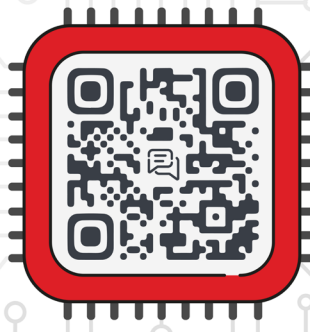




6



5



750 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

Inner & Nested Classes

د. فراس ضعيف
د. معتر النجرس



البرمجة 2

RB Informatcs; 17/05/2026

مدخل إلى مفهوم الكلاسات الداخلية (Introduction)

تناولت المحاضرات السابقة علاقات الأغراض مثل التجميع (Association) والتركيب (Composition)، وظهر من خلالها أن بعض الأصناف، مثل كلاس المحرك Engine أو الدولاب Wheel، يمكن بناؤها كأصناف مستقلة خارج كلاس السيارة Car، ثم استخدامها بصورة منفصلة عند الحاجة.

ومع ذلك، توجد في الهندسة البرمجية حالات تمثل كيانات لا يكون وجودها منطقياً خارج نطاق كيان أكبر يحتويها.

- أمثلة من الواقع: لا يمكن تصور وجود "غرفة معيشة" (Room) مستقلة في الهواء دون وجود "بيت" (House) يحتويها، كما لا يمكن تصور وجود "صف مدرسي" (Classroom) دون وجود "مدرسة" (School) ينتمي إليها.

الغرض الأساسي من الـ Inner Class:

يستخدم هذا الأسلوب عندما تكون الحاجة إلى منع إنشاء غرض من الصنف الداخلي بصورة مستقلة، بحيث لا يوجد هذا الغرض إلا مرتبطاً بوجود غرض من الصنف الخارجي الأكبر.

أنواع الكلاسات الداخلية والمتداخلة (Types of Inner and Nested Classes)

في لغة Java، يجب التمييز بين مصطلحين أساسيين:

- الكلاسات الداخلية (Inner Classes): وهي الكلاسات غير الساكنة التي تُعرّف داخل كلاس آخر، وترتبط عادةً بكائن من الكلاس الخارجي.
- الكلاسات المتداخلة الساكنة (Static Nested Classes): وهي كلاسات تُعرّف داخل كلاس آخر باستخدام الكلمة المفتاحية static، لكنها لا تُعدّ Inner Classes بالمعنى الاصطلاحي الدقيق؛ لأنها لا ترتبط بكائن من الكلاس الخارجي.

وبناءً على ذلك، يمكن تقسيم الكلاسات المعرفة داخل كلاس آخر في Java إلى أربعة أشكال رئيسية:

1. الكلاسات الداخلية العضوية (Member Inner Classes): وهي كلاسات غير ساكنة تُعرّف على مستوى أعضاء الكلاس الخارجي، وهي محور هذه المحاضرة.
2. الكلاسات المتداخلة الساكنة (Static Nested Classes): وهي كلاسات ساكنة تُعرّف داخل كلاس آخر لأغراض تنظيمية غالباً، وهي أيضاً محور هذه المحاضرة.

3. الكلاسات الداخلية المحلية (Local Inner Classes): وهي كلاسات غير ساكنة تُعرّف داخل ميثود أو بلوك معيّن.
4. الكلاسات الداخلية المجهولة (Anonymous Inner Classes): وهي كلاسات غير ساكنة تُنشأ دون اسم، غالباً لتنفيذ واجهة أو توريث كلاس بصورة مختصرة.

الكلاسات الداخلية العضوية (Member Inner Classes)

هو صنف يتم تعريفه داخل نطاق صنف آخر (Outer Class) دون استخدام الكلمة المفتاحية static، ولذلك يعامل معاملة الأعضاء العادية داخل الصنف الخارجي.

ميزته الجوهرية (طلاحيات الوصول):

يمتلك الكلاس الداخلي العضوي صلاحية وصول واسعة؛ إذ يستطيع الوصول مباشرة إلى جميع خصائص وتوابع الصنف الخارجي، حتى وإن كانت معرفة بصلاحية خاصة (private).

آلية الاستدعاء والإنشاء في الدالة الرئيسية (main):

بما أن الصنف الداخلي مرتبط بوجود الصنف الخارجي، فلا يمكن استدعاء باني الصنف الداخلي بصورة مباشرة. يلزم أولاً إنشاء كائن من الصنف الخارجي، ثم استخدام هذا الكائن لإنشاء كائن من الصنف الداخلي وفق الصيغة الآتية:

```
OuterClass outerObj = new OuterClass();
OuterClass.InnerClass innerObj = outerObj.new InnerClass();
```

أمثلة برمجية تطبيقية على الـ Member Inner Class

1. المثال الأول: نظام البيت والغرفة (House & Room)

يوضح هذا المثال كيفية إنشاء كلاس الغرفة داخل كلاس البيت، مع تمكين الغرفة من الوصول المباشر إلى خاصية العنوان address التابعة للبيت.

```
class House {
    String address = "123 Main Street";

    class Room {
        String roomType;

        Room(String type) {
            this.roomType = type;
        }

        void describe() {
            System.out.println(roomType + " in house at " + address);
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        House h1 = new House();
        House.Room r1 = h1.new Room("Bedroom");
        House.Room r2 = h1.new Room("Living Room");

        House h2 = new House();
        House.Room r3 = h2.new Room("Kitchen");

        r1.describe();
        r2.describe();
        r3.describe();
    }
}
```

2. المثال الثاني: نظام السيارة والمحرك (Car & Engine)

يعرض هذا المثال إمكانية إنشاء كائن من الكلاس الداخلي من داخل الكلاس الخارجي نفسه مباشرة، دون الحاجة إلى الصيغة المستخدمة في الدالة main. كما يبين قدرة الكلاس الداخلي على قراءة المتغيرات المعرفة بصلاحيته داخل الكلاس الخارجي.

```
class Car {
    private String model;
    private int fuelLevel;

    Car(String model, int fuel) {
        this.model = model;
        this.fuelLevel = fuel;
    }
    class Engine {
        private int horsepower;
        Engine(int hp) {
            this.horsepower = hp;
        }
        void start() {
            if (fuelLevel > 0) {
                System.out.println(model + " engine started! HP: " +
horsepower);
            } else {
                System.out.println(model + " has no fuel!");
            }
        }
    }
}
```

```
void drive() {
    Engine engine = new Engine(200);
    engine.start();
    System.out.println("Driving the " + model);
}
}
```

3. المثال الثالث: نظام المدرسة والصف (School & Classroom)

يمثل هذا المثال حالة دراسية تؤكد فكرة تقييد إنشاء الصف المدرسي بحيث يكون تابعاً لمدرسة محددة، بدلاً من السماح بوجود صف منفصل في الذاكرة دون مدرسة ينتمي إليها.

```
class School {
    private String schoolName = "Green Valley School";
    private int totalStudents = 500;

    class Classroom {
        private String className;
        private int students;

        Classroom(String name, int students) {
            this.className = name;
            this.students = students;
        }

        void showInfo() {
            System.out.println("School: " + schoolName);
            System.out.println("Class: " + className);
            System.out.println("Students in class: " + students + " / " + totalStudents + " total");
        }
    }
}
```

الكلاسات المتداخلة الساكنة (Static Nested Classes)

عند إضافة الكلمة المفتاحية static إلى كلاس مُعرّف داخل كلاس آخر، يصبح هذا الكلاس كلاساً متداخلاً ساكناً (Static Nested Class)، ولا يُعدّ Inner Class بالمعنى الاصطلاحي الدقيق في Java؛ لأن الـ Inner Class يشير عادةً إلى الكلاسات الداخلية غير الساكنة المرتبطة بكائن من الكلاس الخارجي.

متى نستخدم الـ Static Nested Class؟

يُستخدم هذا النوع عندما تكون الحاجة إلى وضع كلاس داخل كلاس آخر لأغراض تنظيمية فقط، كما في مكتبات الأدوات المساعدة أو الكلاسات الخدمية المرتبطة منطقياً، دون أن يعتمد الكلاس المتداخل على كائنات (Instances) من الكلاس الخارجي.

1. لا يستطيع الكلاس المتداخل الساكن الوصول مباشرة إلى عناصر الكلاس الخارجي غير الساكنة (Non-static members)، لكنه يستطيع الوصول إليها إذا امتلك مرجعاً لكائن من الكلاس الخارجي.
2. يستطيع الوصول مباشرة إلى عناصر الكلاس الخارجي الساكنة (static) حتى لو كانت معرفة بصلاحيات private.
3. يمكن إنشاء غرض منه في الدالة main مباشرة دون الحاجة إلى إنشاء كائن من الكلاس الخارجي.

مقارنة منهجية: الـ Member الكلاسات العضوية مقابل الـ Static الساكنة

تلخص المقارنة الآتية أبرز الفروقات الجوهرية بين الكلاس الداخلي العضوي والكلاس المتداخل الساكن:

وجه المقارنة	Member Inner Class	Static Nested Class
الارتباط بغرض الأب	إلزامي: يرتبط ارتباطاً وثيقاً بوجود غرض (Instance) من الكلاس الخارجي.	مستقل: لا يرتبط بأي غرض من الكلاس الخارجي.
طريقة الإنشاء في الـ main	يتطلب كائناً خارجياً: outer.new Inner()	يتم إنشاؤه عبر اسم الكلاس الخارجي مباشرة: new Outer.StaticInner()
صلاحيات الوصول	يستطيع الوصول إلى عناصر الأب بما فيها العناصر من النوع Private و non-static	يستطيع الوصول مباشرة إلى عناصر الأب الساكنة (static) فقط، ولا يصل مباشرة إلى العناصر العادية غير الساكنة إلا من خلال مرجع لكائن من الكلاس الخارجي.
الهدف البرمجي	نمذجة علاقات واقعية غير قابلة للانفصال، مثل غرفة داخل بيت.	تنظيم الكود البرمجي وبناء مكتبات أدوات مساعدة مشتركة (Utility Classes).

